

第 4 週：函式、程序、參數與組態

本週一句話：讓同一份程式碼變出不同大小的電路，而且不用複製貼上。

本週指令

雙擊 `env.bat` 開好環境（提示字元要有 `[OSS CAD Suite]`），然後：

```
cd week04_func_param
```

每個範例都是這三條（外加一條檢查），把 `02_param_counter` 換成你要跑的名字：

```
iverilog -o sim.out 02_param_counter.v 02_param_counter_tb.v
echo %ERRORLEVEL%
vvp sim.out
gtkwave 02_param_counter.gtkw
```

`echo %ERRORLEVEL%` 印 0 才是編譯成功——失敗時 iverilog 常常一個字都不印。

本週範例名稱：

- `01_function`
- `02_param_counter`
- `03_generate`
- `04_macro`

下面每個範例的段落，都直接附了它自己的四行指令。

學完你要會什麼

- [] 會用 `function` 把重複的組合邏輯包起來
- [] 分得出 `function` 和 `task` 的差別
- [] ★ 會用 `parameter` 寫可調整大小的模組
- [] ★ 會用 `generate` 複製一整排相同的零件
- [] 會用 ``define` / ``ifdef` 做條件編譯

一、function

```
function [3:0] count_ones;      // [3:0] 是回傳值寬度
    input [7:0] x;              // 輸入參數
    integer i;                  // 可以有區域變數
    begin
        count_ones = 0;         // ★ 用「函式名 = 值」回傳
        for (i = 0; i < 8; i = i + 1)
            count_ones = count_ones + x[i];
    end
endfunction
```

規矩：

項目	function	task
回傳值	一定要有一個	沒有 (用 output 參數)
可以有延遲 #	✗ 不行	✓ 可以
可以呼叫 task	✗ 不行	✓ 可以
能不能合成	✓ 可以 (變組合邏輯)	通常不行 (多半用在 testbench)
典型用途	設計裡的重複運算	測試平台的重複步驟

[!IMPORTANT] `function` 裡的 `for` 迴圈不是跑迴圈，綜合時會被完全展開成硬體。 `count_ones` 的 8 次迴圈 → 8 個加法器並排，一個時脈就算完。

二、★ parameter：可調整大小的模組

```
module param_counter #(
    parameter WIDTH = 4,
    parameter MAX    = 9
)(
    input  wire      clk,
    output reg [WIDTH-1:0] count    // ← 寬度由參數決定
);
```

用的時候指定參數：

```
param_counter #(.WIDTH(4), .MAX(9))  u_bcd (...); // 十進位
param_counter #(.WIDTH(4), .MAX(15)) u_hex (...); // 十六進位
param_counter #(.WIDTH(8), .MAX(20)) u_wide (...); // 0~20
```

同一份程式碼，做出三個大小完全不同的電路。

關鍵字	差別
parameter	外面可以改（實例化時指定）
localparam	只有模組內部用，外面不能改

[!TIP] 業界規範：所有寫死的數字都應該變成 **parameter**。這樣改規格時只要改一行，不用全檔搜尋取代。

三、★ generate：複製硬體

沒有 generate，8 位元加法器要手寫 8 次實例化。有了它：

```

genvar i;                                // ★ generate 專用的迴圈變數
generate
    for (i = 0; i < N; i = i + 1) begin : adder_stage
        full_adder1 fa (
            .a    (a[i]),
            .b    (b[i]),
            .cin   (carry[i]),
            .sum   (sum[i]),
            .cout  (carry[i+1]) // 這級的進位接到下一級
        );
    end
endgenerate

```

[!IMPORTANT] generate 的 **for** 不是「執行迴圈」，是「複製零件」的指示。

綜合時它會展開成 N 顆實體的全加器並排，全部同時運作。 **begin :**
adder_stage 那個標籤是必要的——展開後每一顆會叫
adder_stage[0].fa 、 **adder_stage[1].fa**

改一個 **N**，就從 8 位元變 32 位元。這就是硬體設計的複製貼上。

四、`define 與條件編譯

```

`define DATA_W 8
`define USE_FAST_ADDER // 開關

module foo (input [`DATA_W-1:0] a, ...);

`ifdef USE_FAST_ADDER
    assign result = a + b; // 有定義 → 編這段
`else
    assign result = a - b; // 沒定義 → 編這段
`endif

```

這些是前置處理器指令，在編譯之前就處理完了——硬體完全看不到它們。

⚠️ ` (反引號) 不是單引號。位置在鍵盤左上角 Esc 下面那顆。

五、本週四個範例

```
cd week04_func_param
```

01_function — 三個實用函式

```
iverilog -o sim.out 01_function.v 01_function_tb.v
echo %ERRORLEVEL%
vvp sim.out
gtkwave 01_function.gtkw
```

奇同位、取大值、數 1 的個數。注意 `count_ones` 裡的 `for` 迴圈會被展開成硬體。

02_param_counter — 參數化計數器 ★

```
iverilog -o sim.out 02_param_counter.v 02_param_counter_tb.v
echo %ERRORLEVEL%
vvp sim.out
gtkwave 02_param_counter.gtkw
```

看什麼：波形上三個計數器同時在數，但歸零時機完全不同。

實測：

bcd(0-9)	hex(0-F)	wide(0-20)	
9 *	9	9	← bcd 數到 9 就 tick
0	10	10	
...			
5	15 *	15	← hex 數到 15 才 tick
6	0	16	

程式碼只有一份。這就是 parameter 的價值。

03_generate — 用 generate 做 N 位元加法器 ★

```
iverilog -o sim.out 03_generate.v 03_generate_tb.v
echo %ERRORLEVEL%
vvp sim.out
gtkwave 03_generate.gtkw
```

看什麼：GTKWave 左邊的樹會出現 `adder_stage[0]` ~ `adder_stage[7]` 八個實例。點進任一個可以看那一位的 `cin`、`cout`。

這就是你第 1 週練習 2 的完整版，而且測了 200 組亂數全過。

實測：

```
200 + 100 + 0 = cout=1 sum=44 < 300 超過 255，進位出去了
>>> 含 200 組亂數測試，全部正確！
```

04_macro — 巨集與條件編譯

```
iverilog -o sim.out 04_macro.v 04_macro_tb.v
echo %ERRORLEVEL%
vvp sim.out
gtkwave 04_macro.gtkw
```

動手試：把 `04_macro.v` 裡的 ``define USE_FAST_ADDER` 註解掉，重跑——加法會變成減法，同一份原始碼編出不同電路。

六、練習題

練習 1

把第 1 週的半加器改寫成 `function`，然後用它做全加器。

練習 2 ★

寫一個參數化的格雷碼計數器 `gray_counter #(parameter W = 4)`：輸出相鄰兩個值只差一個位元。

提示：先做一般二進位計數器 `bin`，再 `gray = bin ^ (bin >> 1)`。

練習 3 ★

用 `generate` 寫一個 `N` 位元的優先權編碼器：輸入 `N` 位元，輸出「最高位的 1 在第幾位」。

練習 4 (思考)

下面這樣寫，會產生幾個加法器？

```
integer i;
always @(*) begin
    sum = 0;
    for (i = 0; i < 8; i = i + 1)
        sum = sum + data[i];
end
```

► 看答案

七、本週檢核

- ☐ 四個範例都跑過
- ☐ 02 的波形能指出三個計數器歸零時機不同
- ☐ 03 在 GTKWave 樹狀圖找得到 `adder_stage[0..7]`
- ☐ 04 動手把 `define` 註解掉試過一次
- ☐ 練習 2 寫出來
- ☐ 練習 4 答對 (答錯就再讀一次「generate 是複製零件」那段)

附註：關於 `sim` 這個指令

你可能在別的地方看過我寫的 `sim` 範例名稱。那是我自己做的批次檔 (`sim.bat`)，不是 Verilog 或 iverilog 的標準指令，教科書上查不到。

它做的事就是把上面那四行包起來、順便幫你檢查結束碼。用不用都可以，也可以刪掉。這份 README 裡給的全部都是原始指令。